Zürcher Hochschule
für Angewandte Wissenschaften

School of
Engineering
InES Institute of
Embedded Systems

Embedded Software Engineering

Simon Künzli, Tobias Welti

HS 2025

Agenda HS25

Zürcher Hochschule für Angewandte Wissenschaften
School of Engineering
InES Institute of Embedded Systems

Vorlesung (welo)
Vorlesung (kuex)
Übungen / Praktika

Stand: 27.08.25

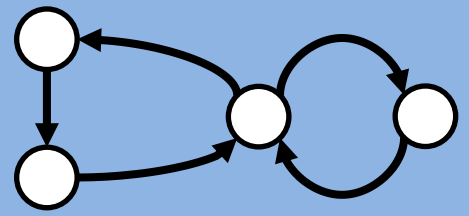
wk #	Do, 10:00 - 11:35	Do, 08:00 – 09:35 / Do 12:00 – 13:35	
01 15.09.	Modulorganisation & Einführung Embedded Systems	No Lab	
02 22.09.	SW-Paradigmen für Embedded Systems	No Lab	P1: First steps U96
03 29.09.	Requirements für Embedded Systems	P1: First steps U96	P1: Finish Lab
04 06.10.	Software-Entwicklung für ES (Modellierung, SW-Specs)	P1: Finish Lab	U1: Requirements
05 13.10.	Entwicklungsprozesse / CI/CD	U1: Requirements	U2: Modeling
06 20.10.	Non-functional Requirements	U2: Modeling	U3: Implementation & Test
07 27.10.	Visite Fertigung Siemens	Visite Fertigung Siemens	
08 03.11.	Fokus "Energie« / andere Prozessoren?	U3: Implementation & Test	P2: Energy Consumption
09 10.11.	Fokus "Performance"	P2: Energy Consumption	P3: AES in SW, incl. Measurement on R5
10 17.11.	Einführung FPGA	P3: AES in SW, incl. Measurement on R5	P3: Finish Lab
11 24.11.	Einführung Design Space Exploration (DSE)	P3: Finish Lab	P4: AES in FPGA and HW-Accelerator
12 01.12.	DSE: Kostenfunktionen und Suche im Entwurfsraum	P4: AES in FPGA and HW-Accelerator	P4: Finish Lab
13 08.12.	Einführung RTOS	P4: Finish Lab	P5: DSE
14 15.12.	Dual-Prozessor-Systeme mit Uniform Memory Access	P5: DSE	No Lab

Advance Organizer ESE

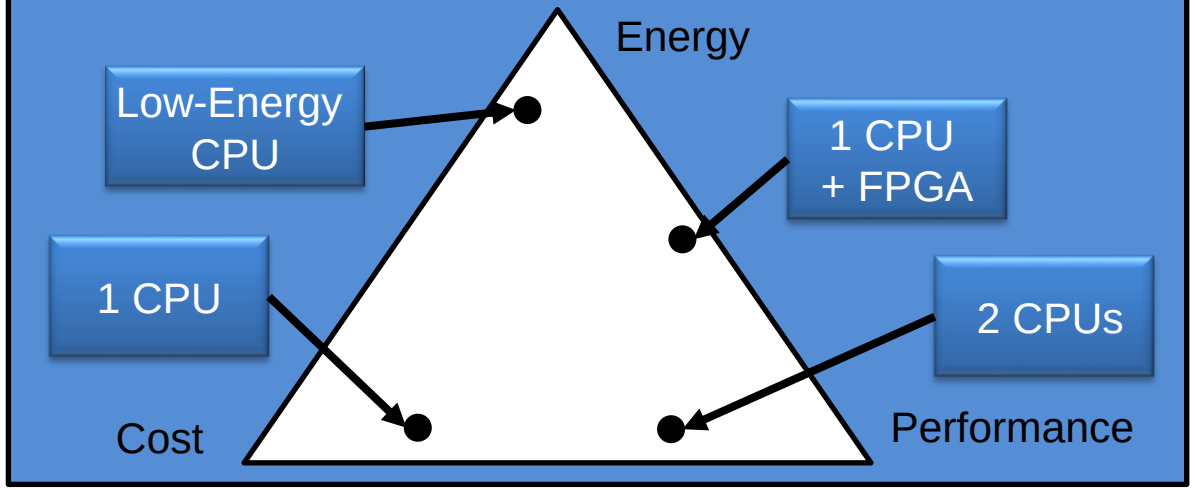
Embedded Systems Engineering

Functional & Non-Functional Requirements

Software & Hardware Modelling

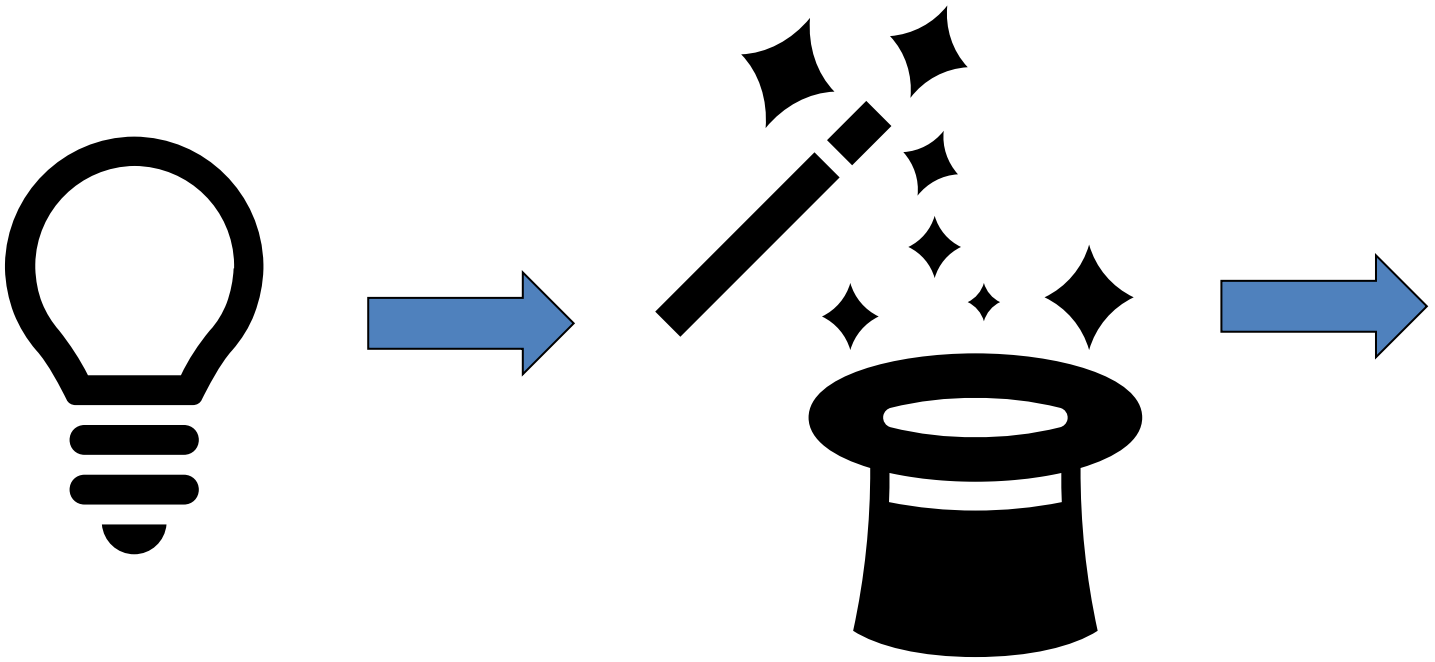


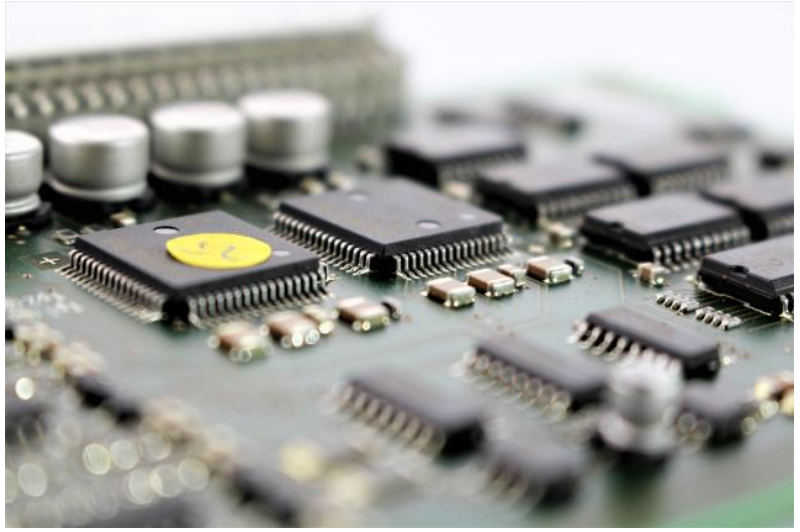
Design Space



Hardware-BOM & Software-Implementation

Motivation



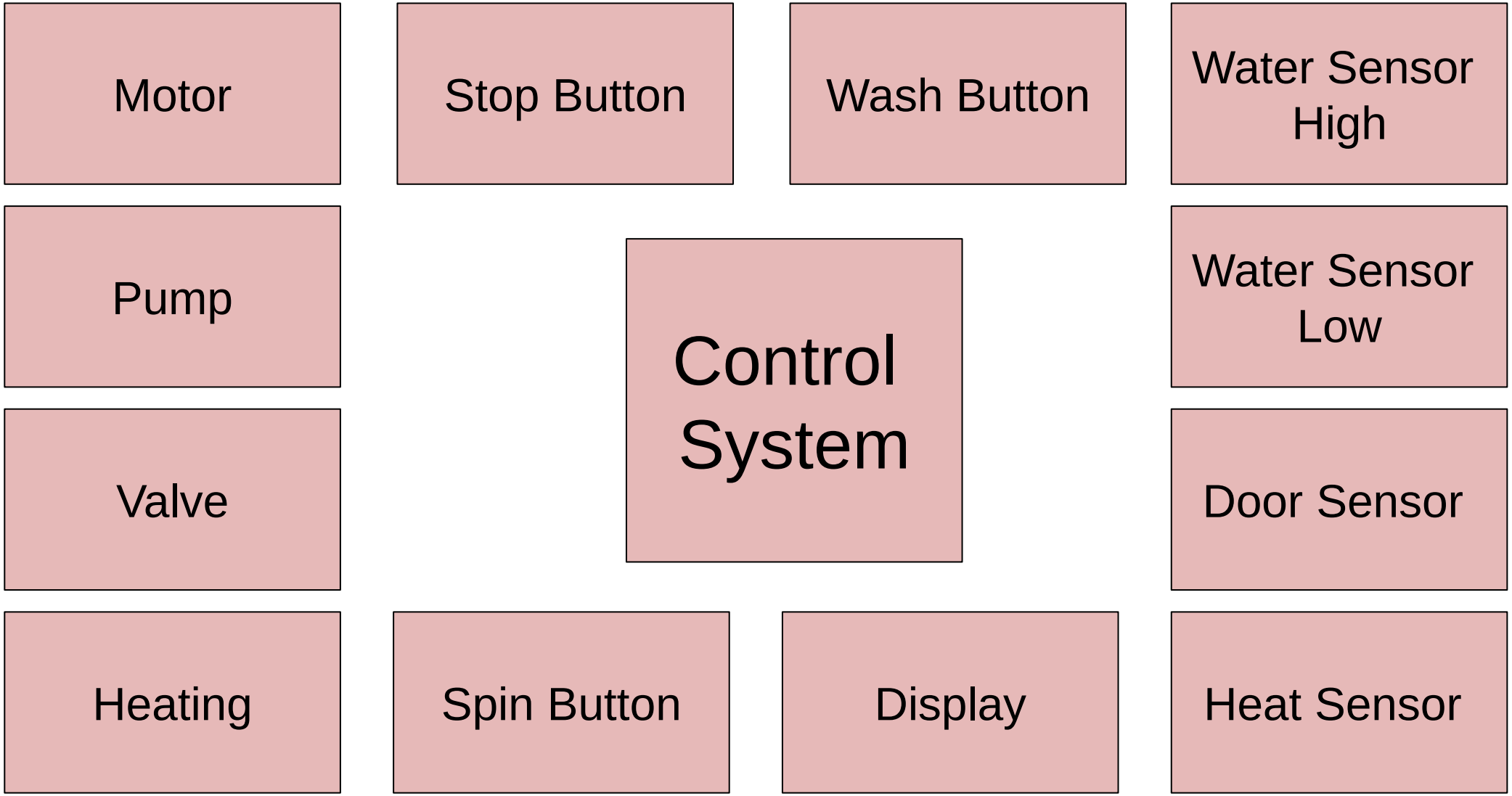


Learning Objectives

After today's lesson, the students…

- **… know several development methods that can be used for Embedded System development**
- **… can name advantages and disadvantages of specific development processes**
- **… know how to select the best-matching development process depending on the task and adapt the process if needed**
- **… know abstraction layers and how they may influence the development**
- **… can name and explain the phases of the waterfall model**
- **… can explain the main goals of an agile development model**

Example Washing Machine – Introducing Abstraction



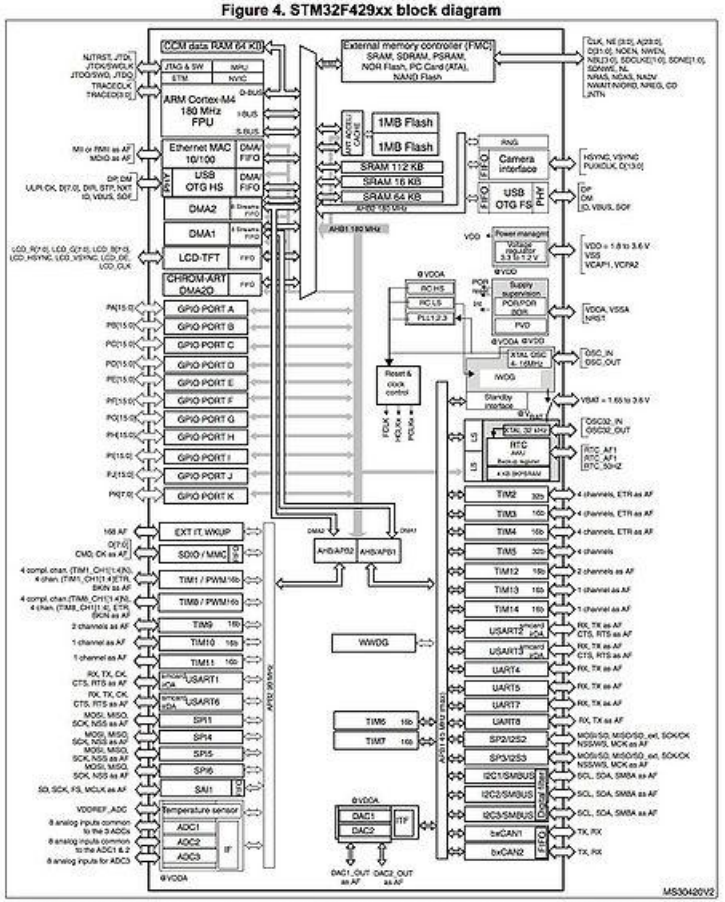
Washing Machine – Inside the Control System

SW-Model

Washing Machine – Possible UML State Machine

From U1 ☺

HW-Model



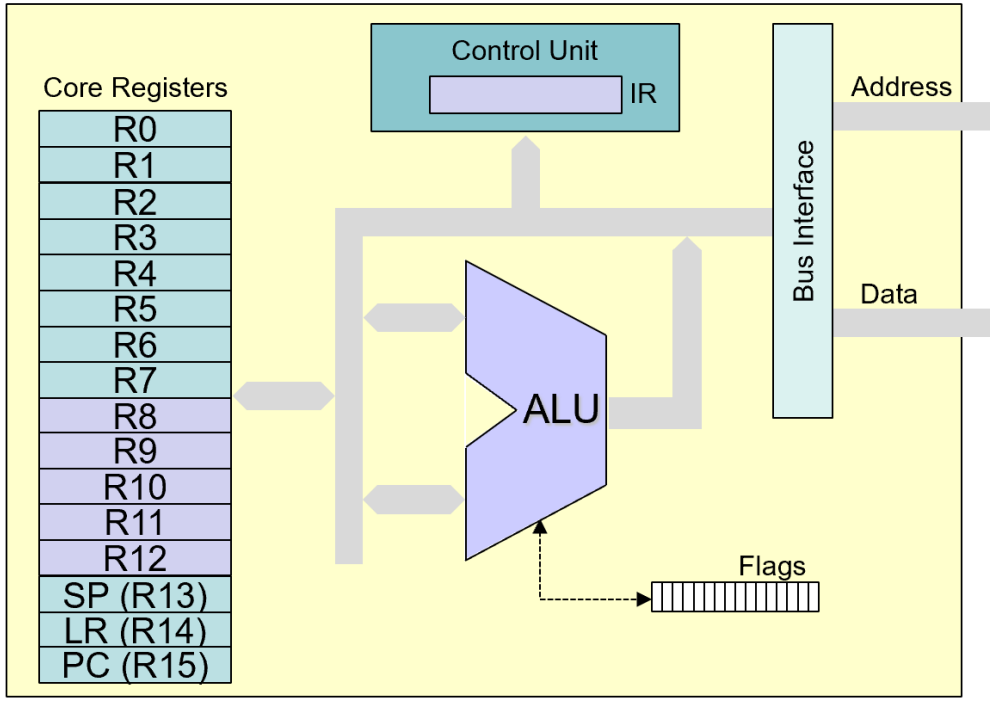
1. The timers connected to APB2 are clocked from TIMxCLK up to 180 MHz, while the timers connected to APB1 are clocked from TIMxCLK either up to 90 MHz or 180 MHz depending on TIMPRE bit configuration in the RCC_DCKCFGR register.

Washing Machine – Deeper Inside the Control System

Software

```
/* ---------------------------------------------
case WASHING_LEFT:
    switch (event) {
        case BUTTON_STOP:
            timer_stop();
            ah_motor_stop();
            ah_pump_on();

            ah_lcd_write(TEXT_STOPPING);
            state = STOPPING;
            break;

        case TIME_OUT:
            ah_motor_stop();
            ah_pump_on();

            ah_lcd_write(TEXT_WATER_OUTTAKE);
            state = WATER_OUTTAKE;
            break;

        default:
            break;
    }
    break;
```

Hardware

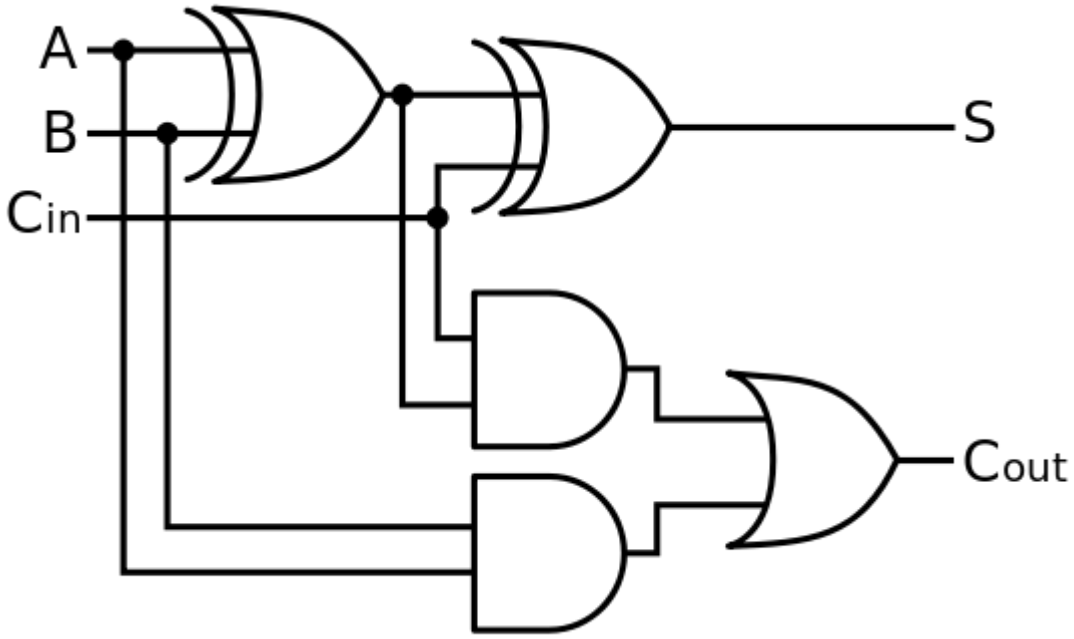


Washing Machine – Further down into Details

- **Software**

```
00000000 20A5
00000002 2111
00000004 1840

00000006 4A00
00000008 6010

0000000A 00002000
```

- **Hardware**

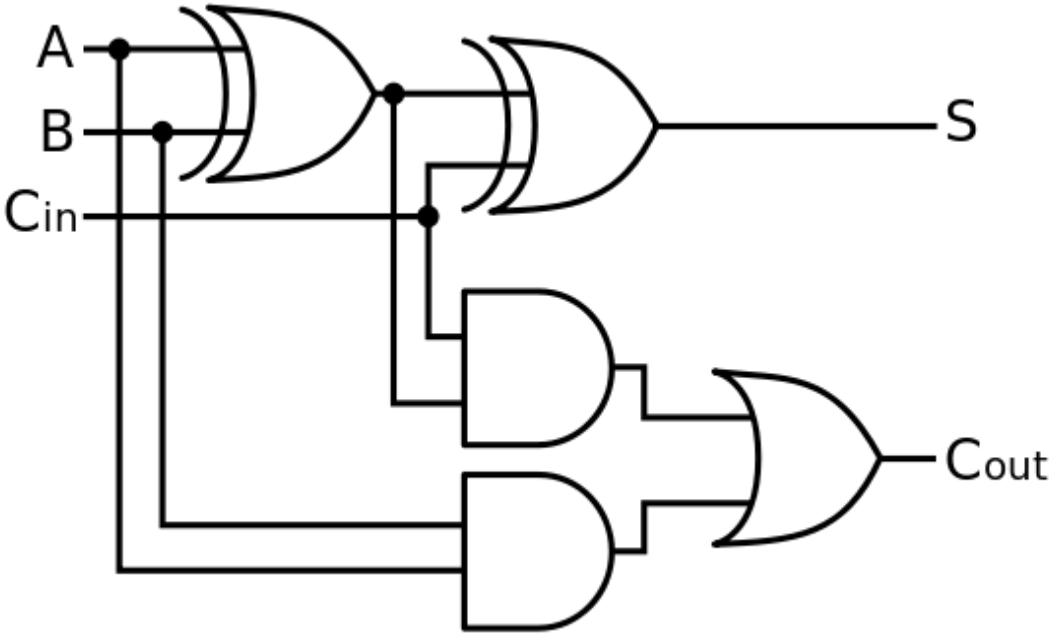


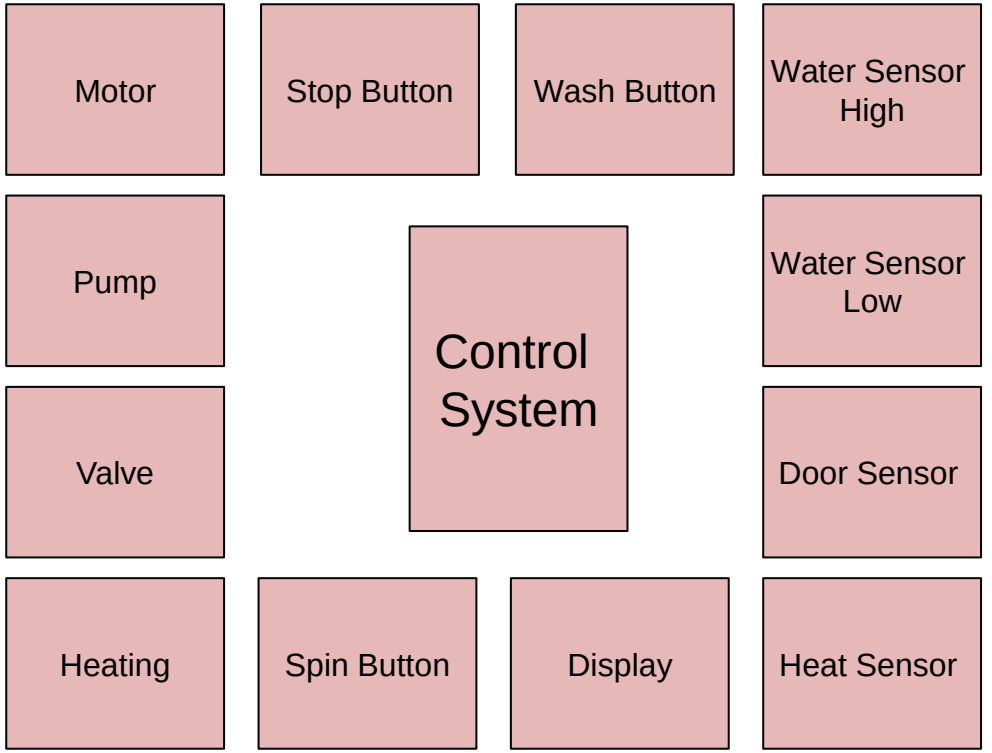
Abstraction Levels

Level	Hardware	Software
System	Components and Sub-systems	Tasks that describe the system functionality
Module	Processors, MCUs, ASICs, Buses, Peripherals, Memory, Storage	Interacting SW modules
Block	Counter, Registers, ALUs	Program sequences, loops
Logic	Logic gates, Flip-Flops	Instructions (e.g. ADDS R1, R2, R3)
Device	Electronic components, transistors	Machine Code (e.g. 0x21ff)

Designing an Embedded System

- **Where should we start?**





Design Process

- **Process to implement a desired functionality (functional requirements) with a specific set of physical components.**

1. Modelling/Specification of the functional behavior

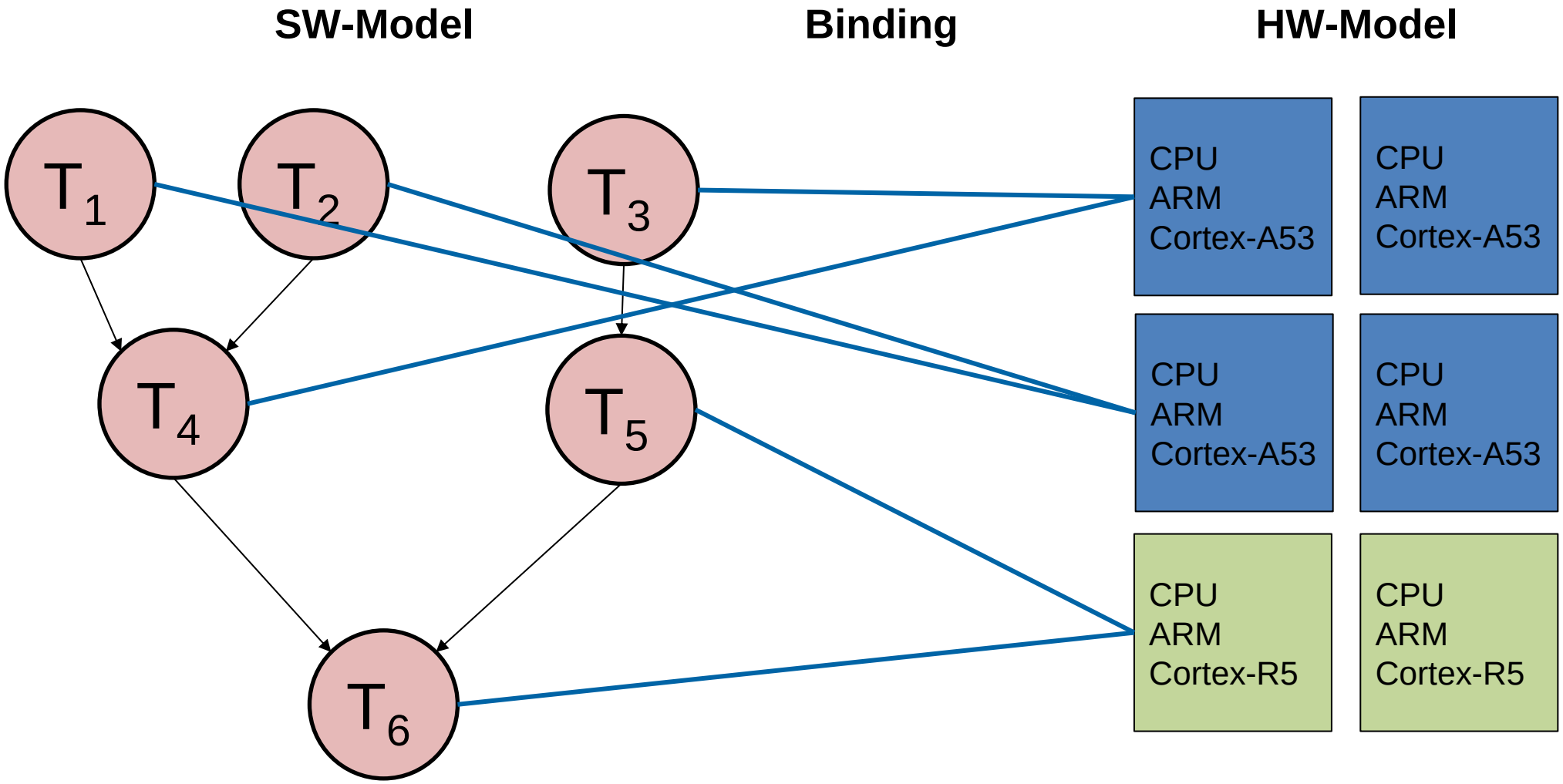
Design Process

1. **Modelling/Specification** of the functional behavior

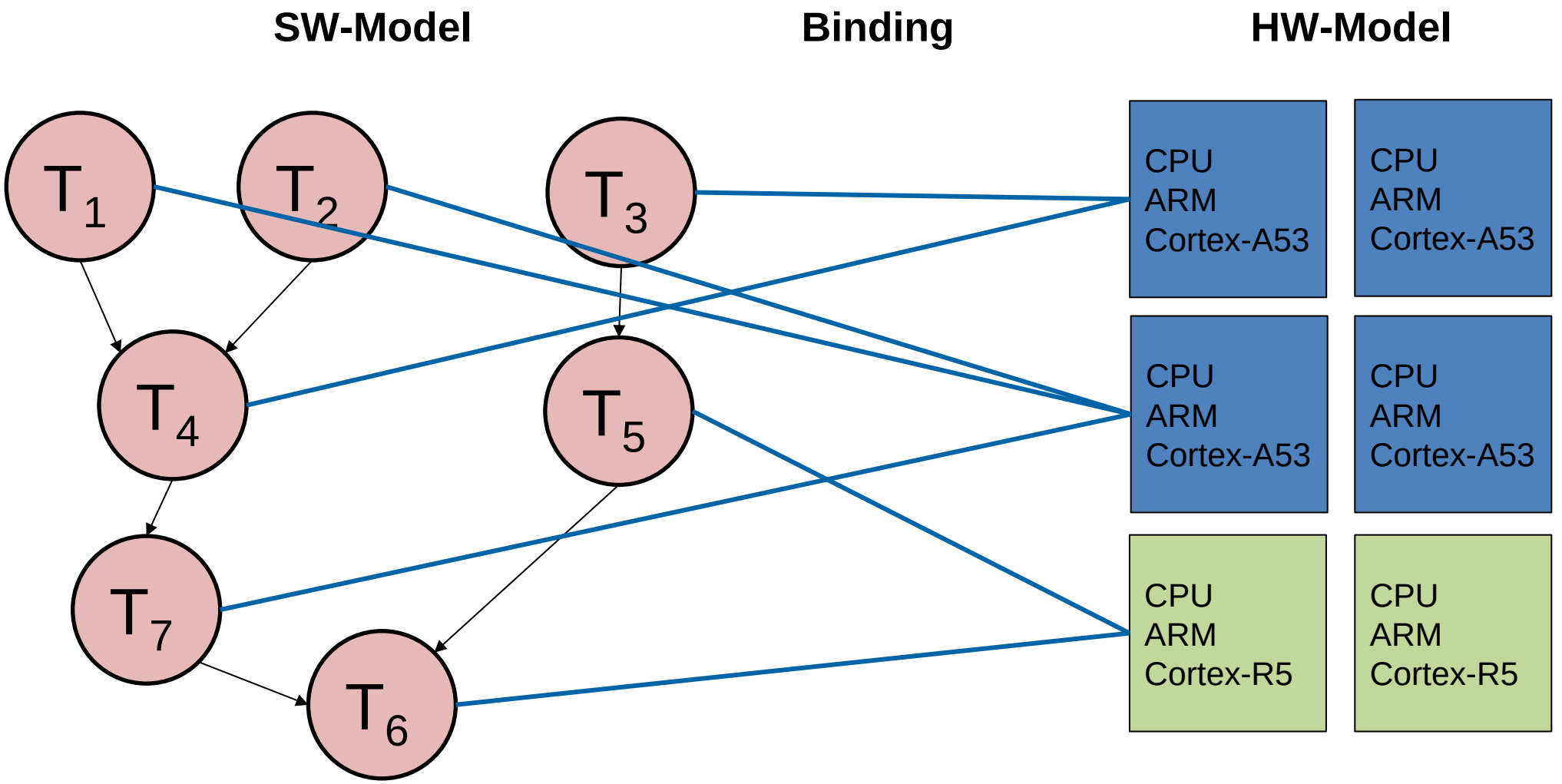
2. **Explore alternatives to the initial specification**
   - Are there different possibilities to specify the desired behavior?
   - Example Washing Machine:
     - Could we start heating and adding water at the same time? Could we simplify the emergency behavior while being safe?
     - Could we add additional states to simplify the specification process?
     - Could we run action handlers on different MCUs?
   - …

3. **Refinement**
   - Decide on the most promising solution and go to the next deeper level of abstraction

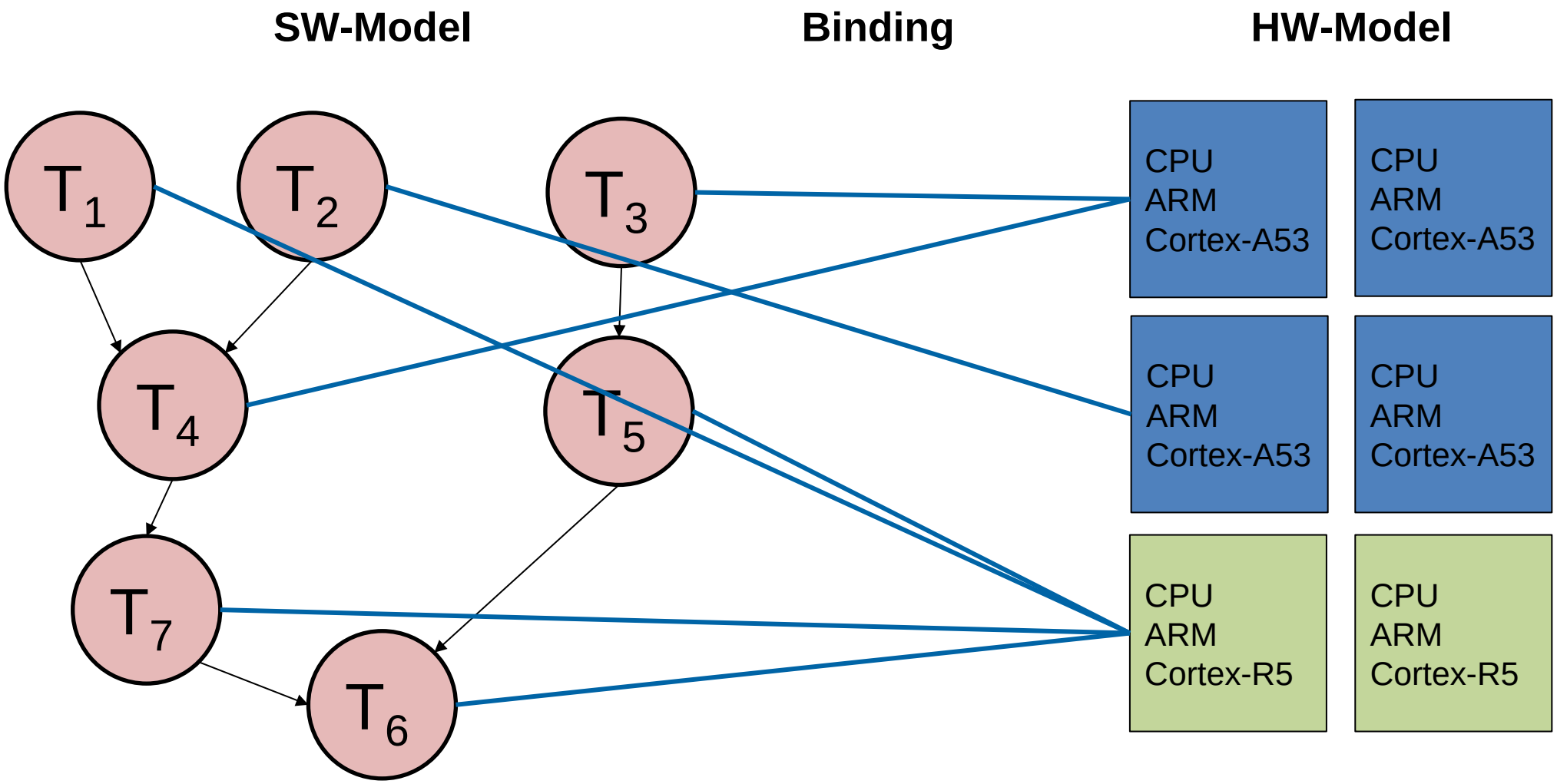
Recap: Embedded System – Multiprocessor System



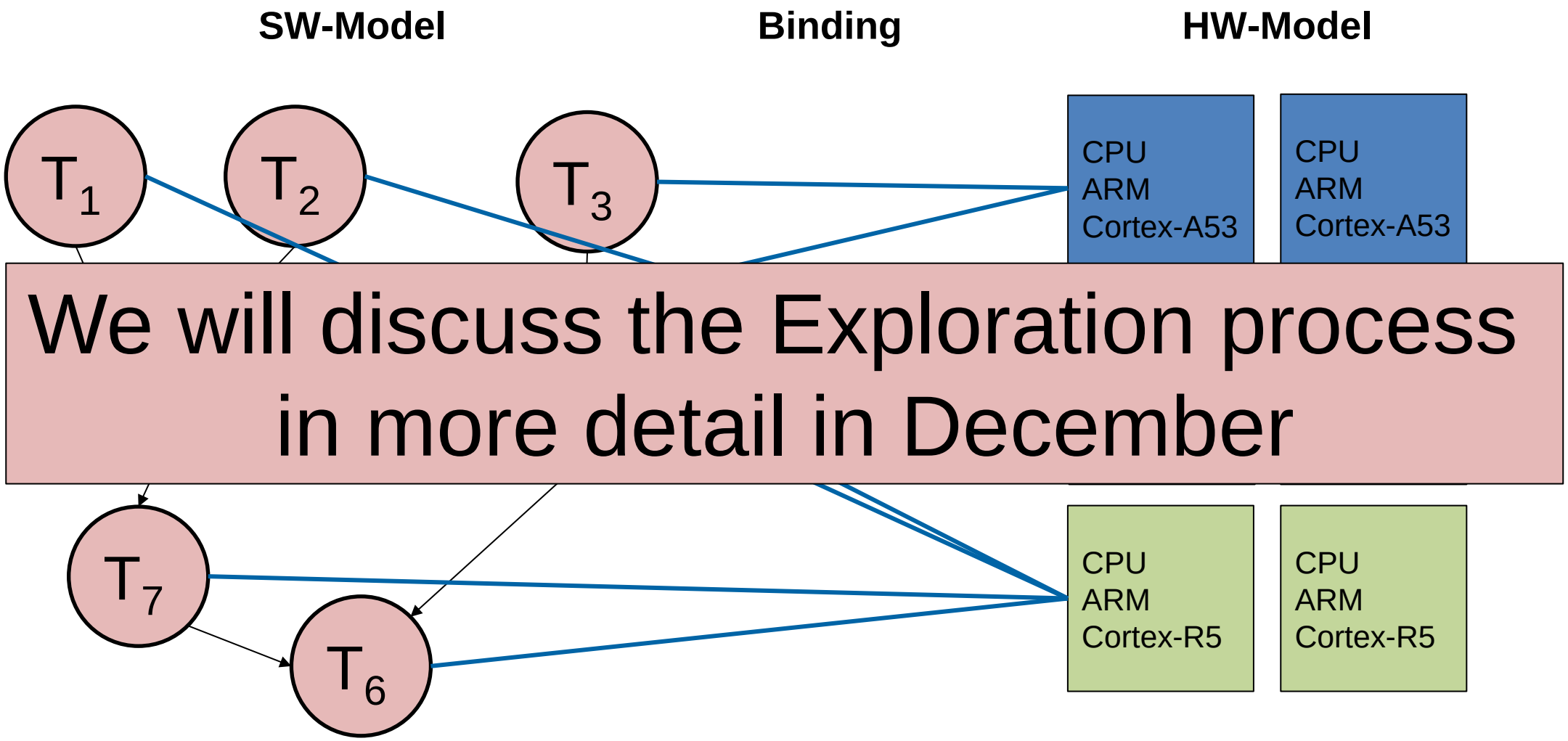
Recap: Embedded System – Multiprocessor System



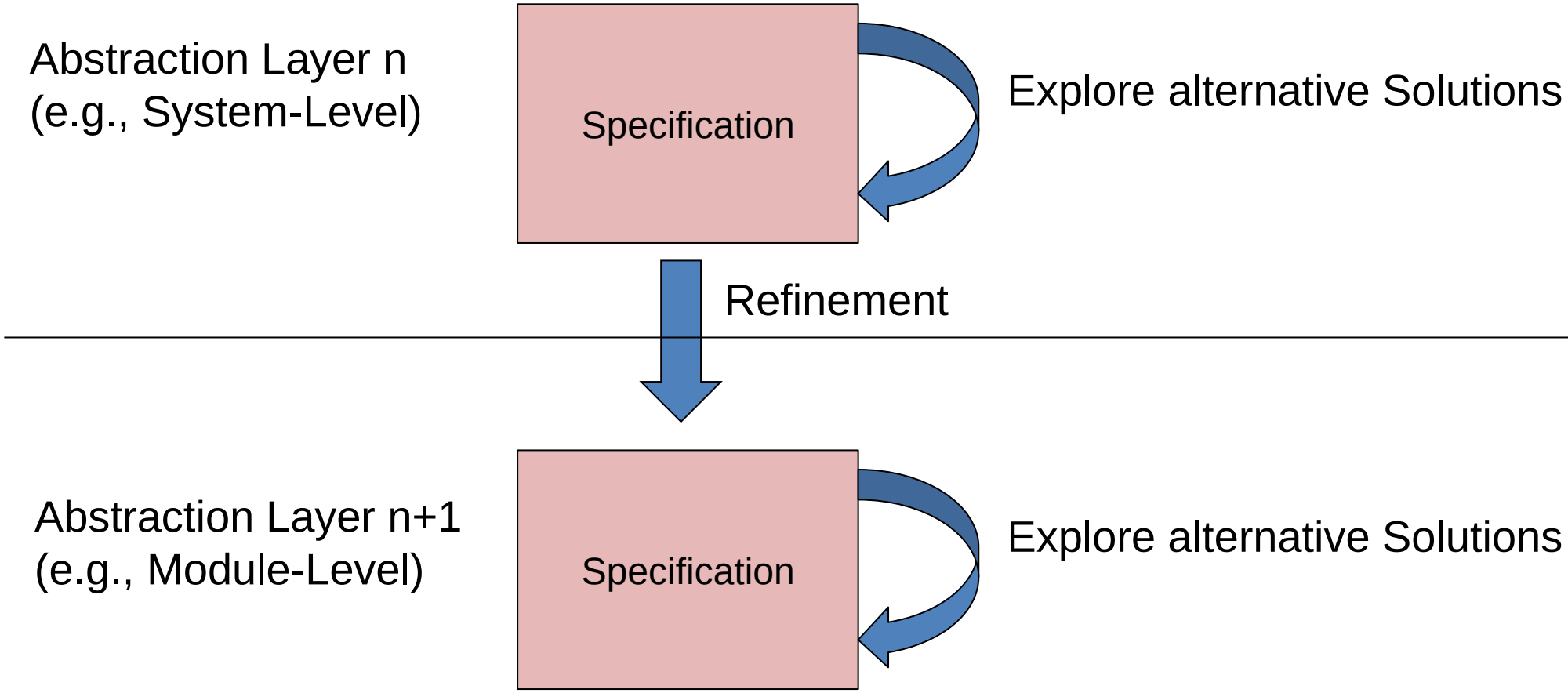
Recap: Embedded System – Multiprocessor System



Recap: Embedded System – Multiprocessor System

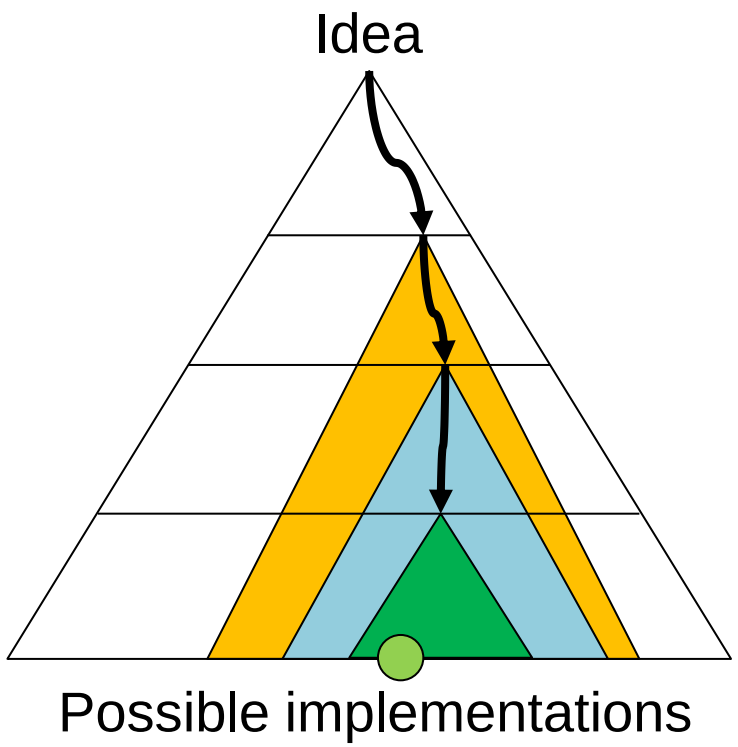


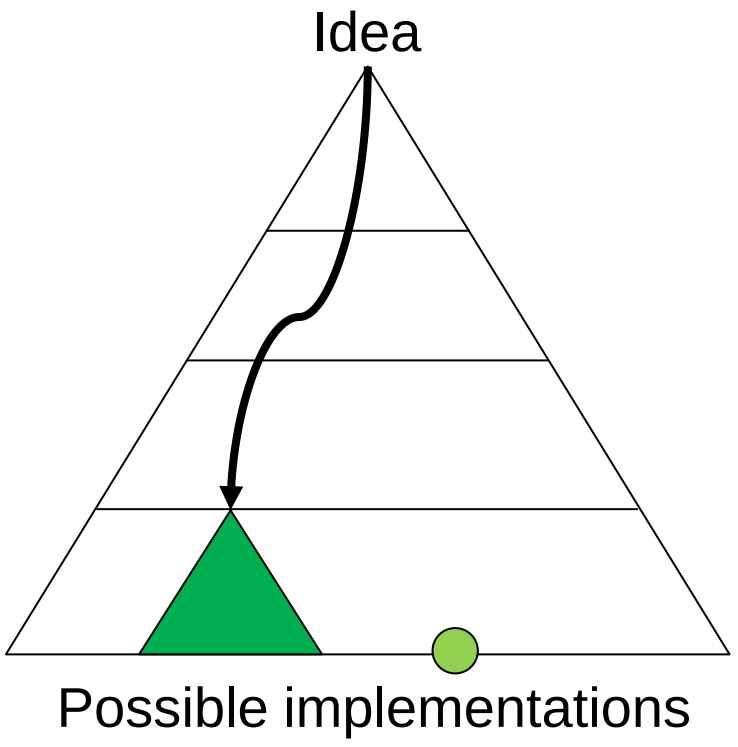
Specification, Exploration, Refinement



Top-Down Design Approach

- **Higher efficiency with higher abstraction layers**
- **Keep solution space wide open as long as possible**
- **If we carefully refine and explore, we should find the optimal solution**





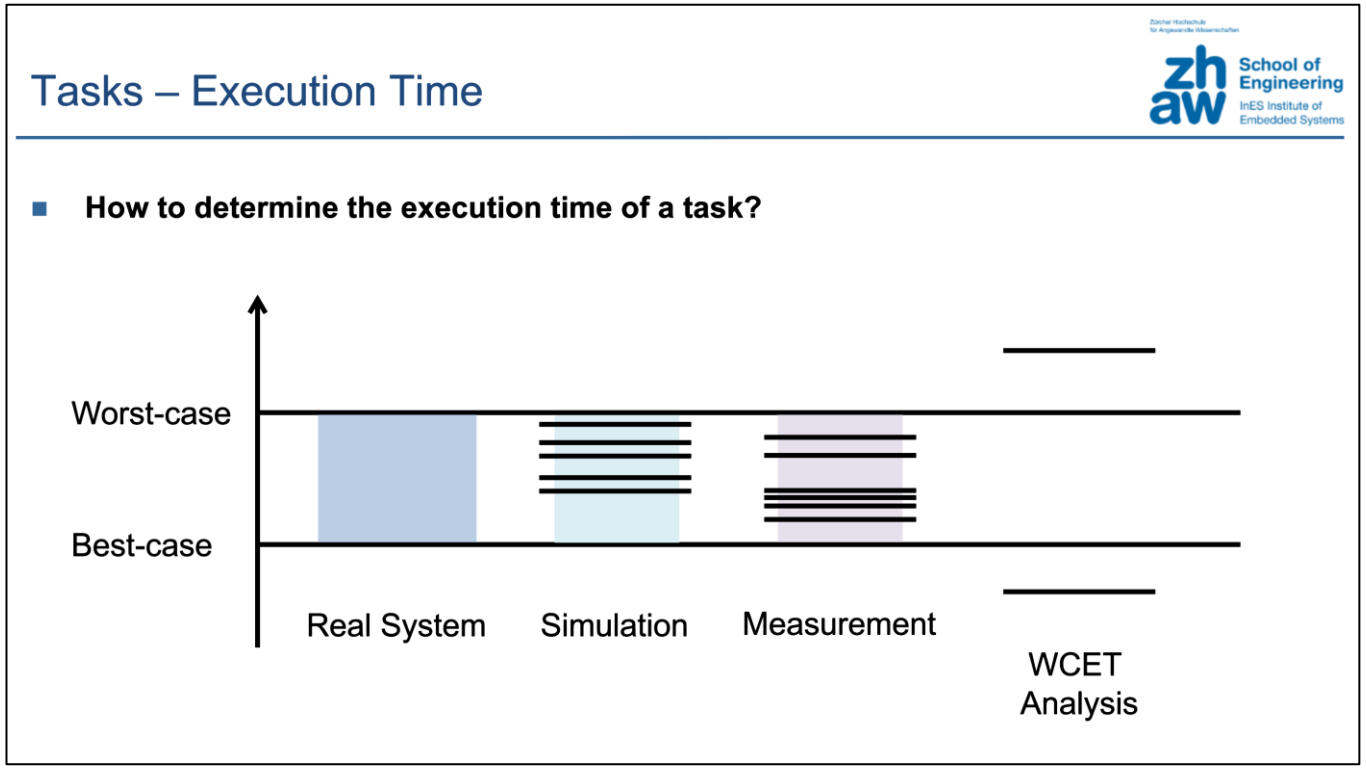
Source: Vahid/Givargis, «Embedded System Design: a Unified HW/SW Introduction»

Verification & Evaluation on differe evels

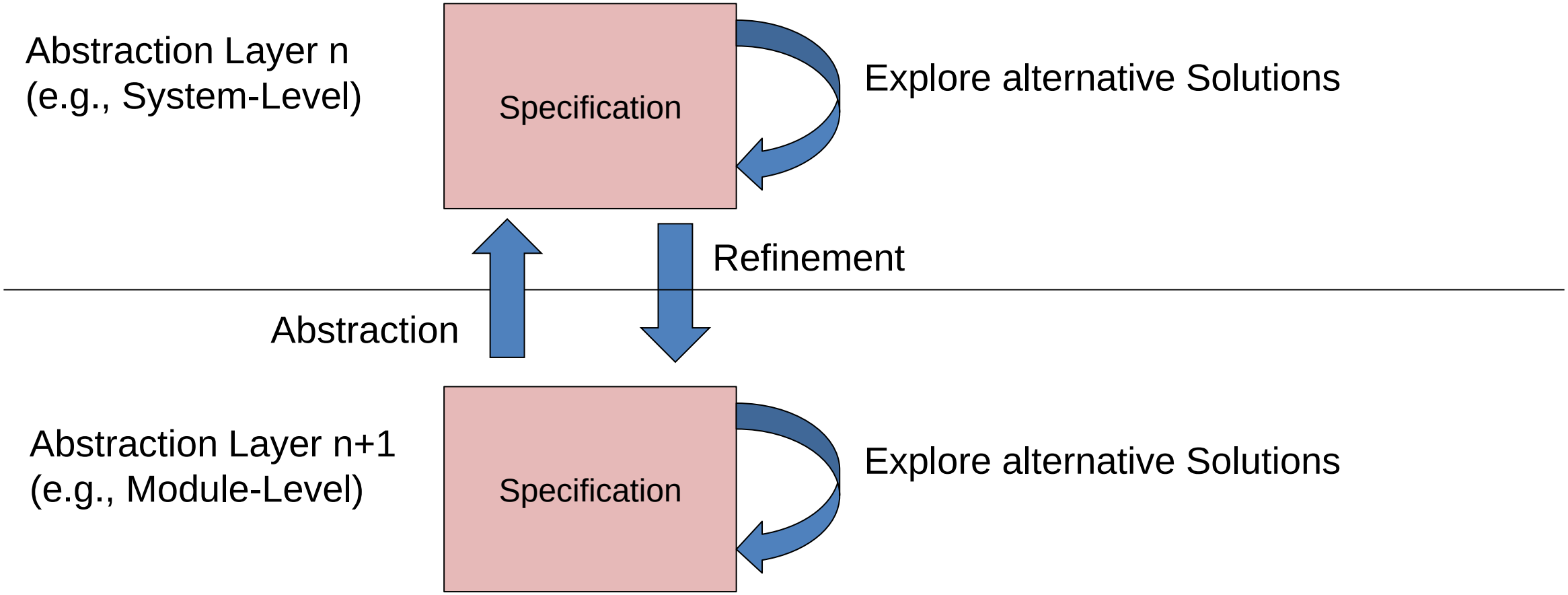
Functional Requirements

Non-Functional Requirements

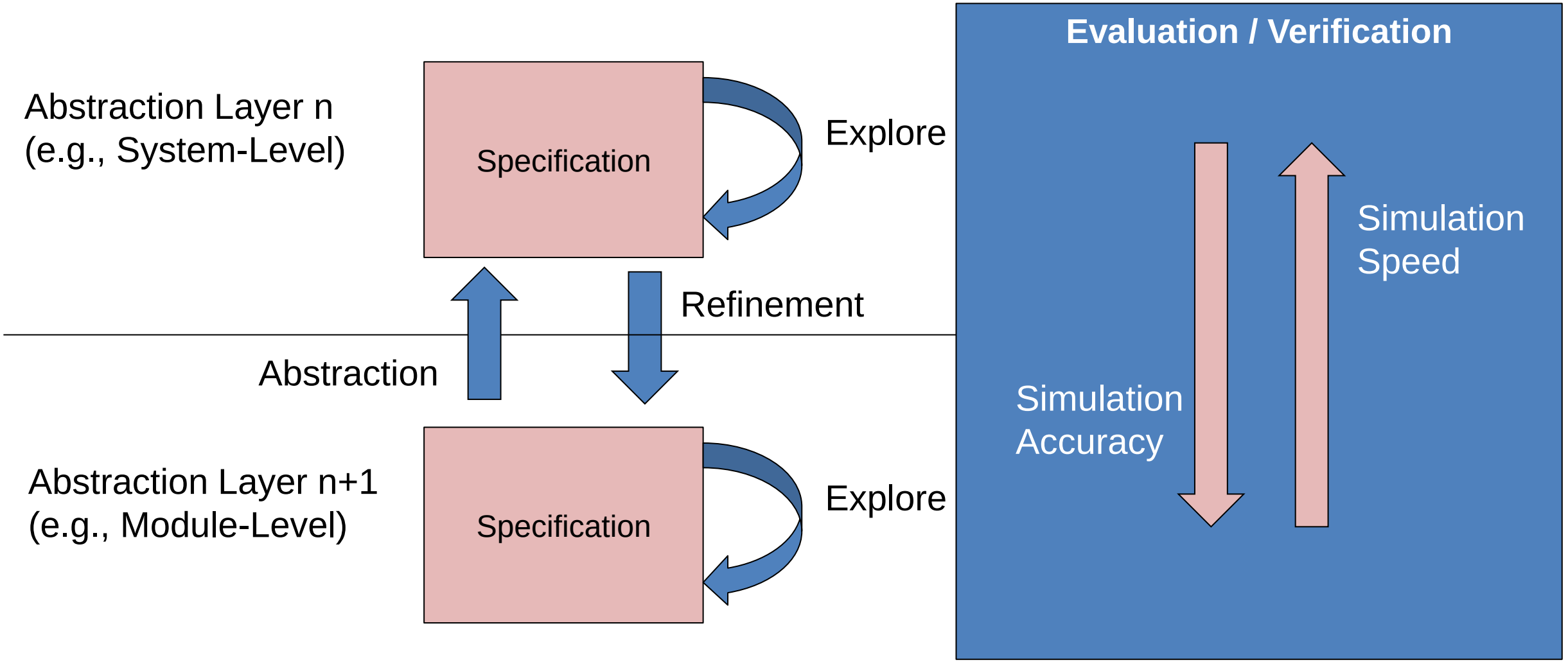
- **Does our system do the right things right?**
  - Formal Model Verification with, e.g., Model Checking
  - Analysis
  - Simulation
  - Measurements



Specification, Exploration, Refinement



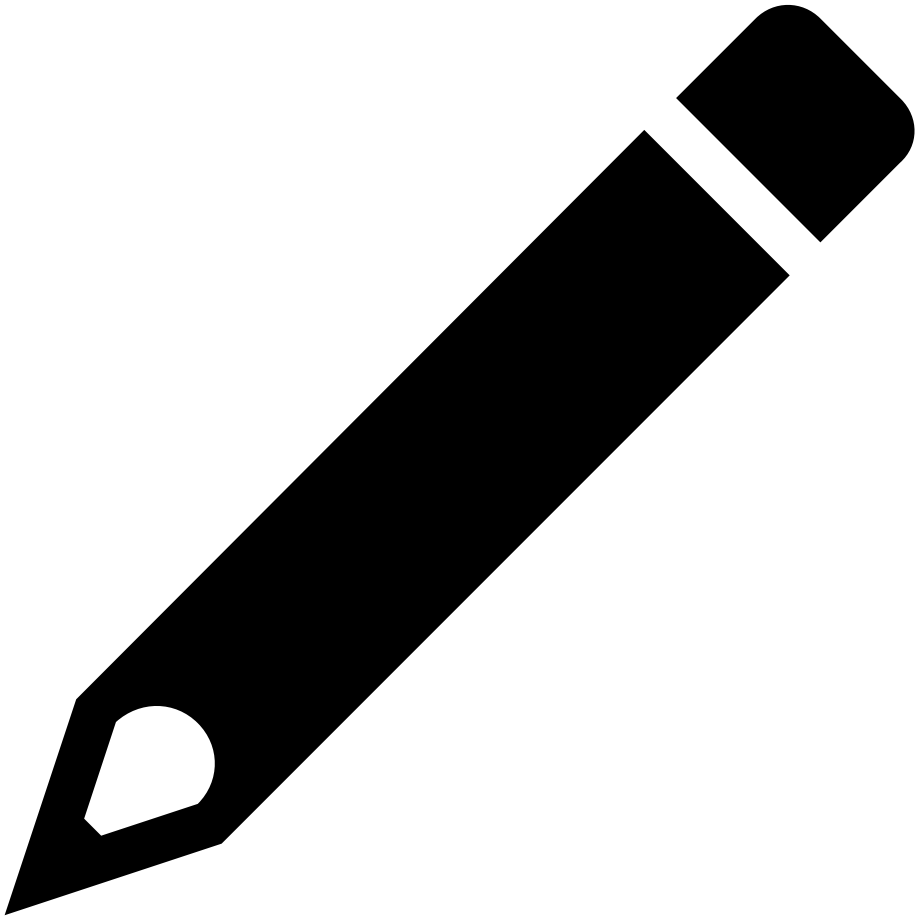
Specification, Exploration, Refinement



Discussion on Abstraction Layers

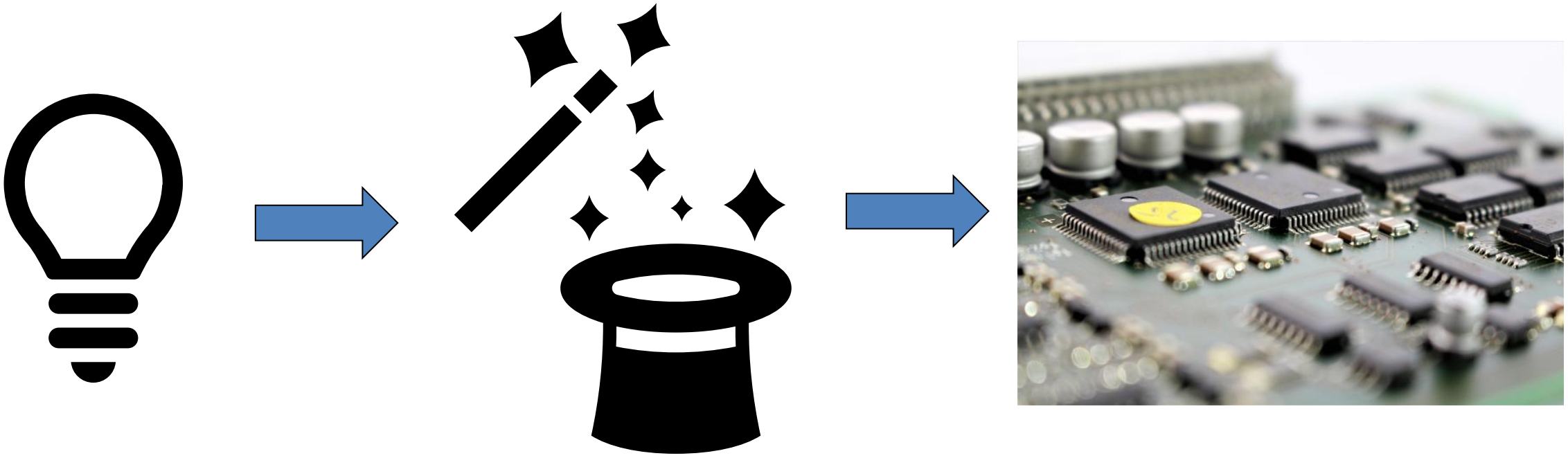
- **Discuss with your neighbor and find examples for the different abstraction layers that we discussed so far for a traffic light control system**

- **Add your ideas to edupad**
  - Link available in Moodle
  - https://edupad.ch/p/rXiBGQU5OJ

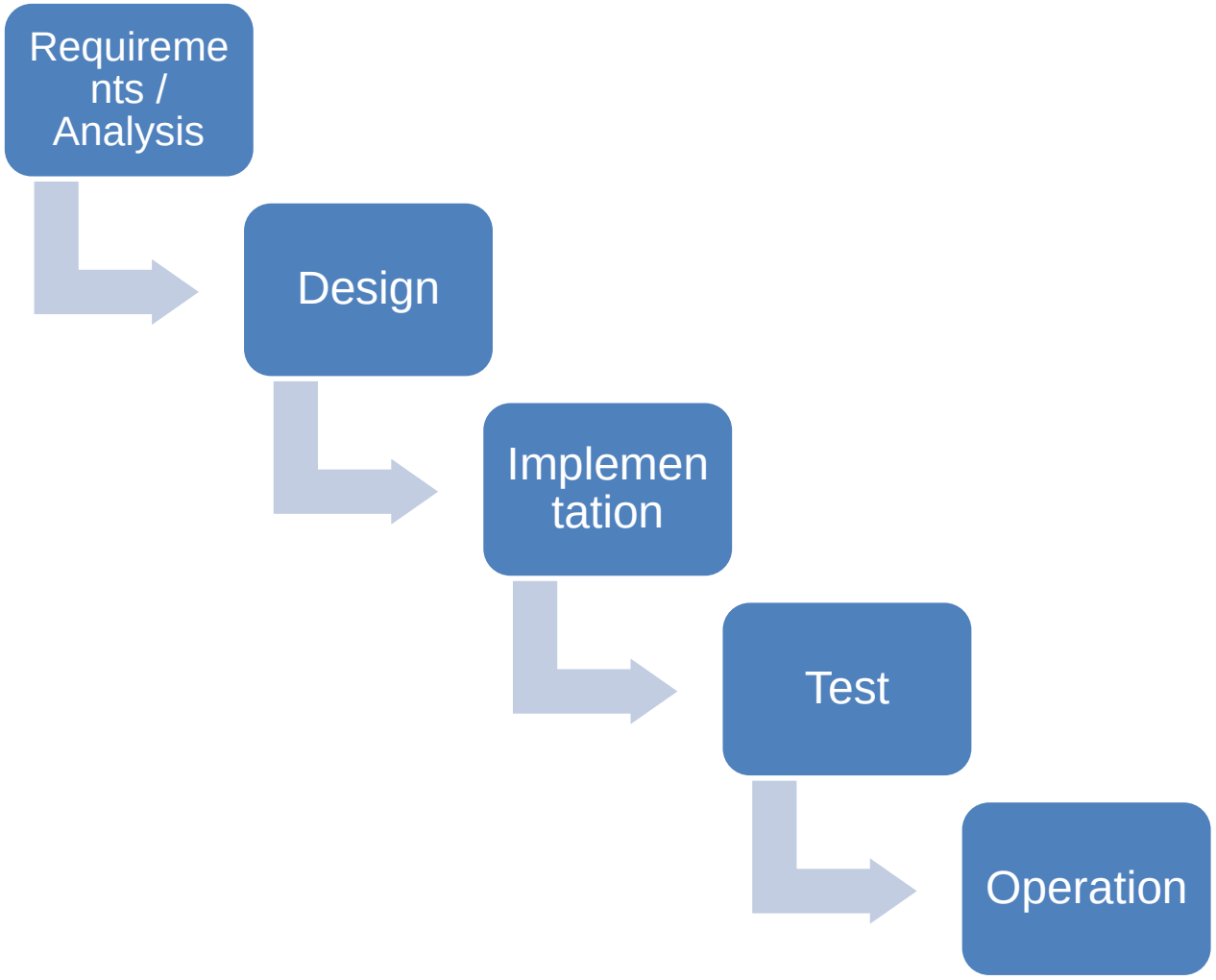


So far, we covered the Design of an Embedded System

- **But how do we proceed during the whole development from idea to product?**



Waterfall



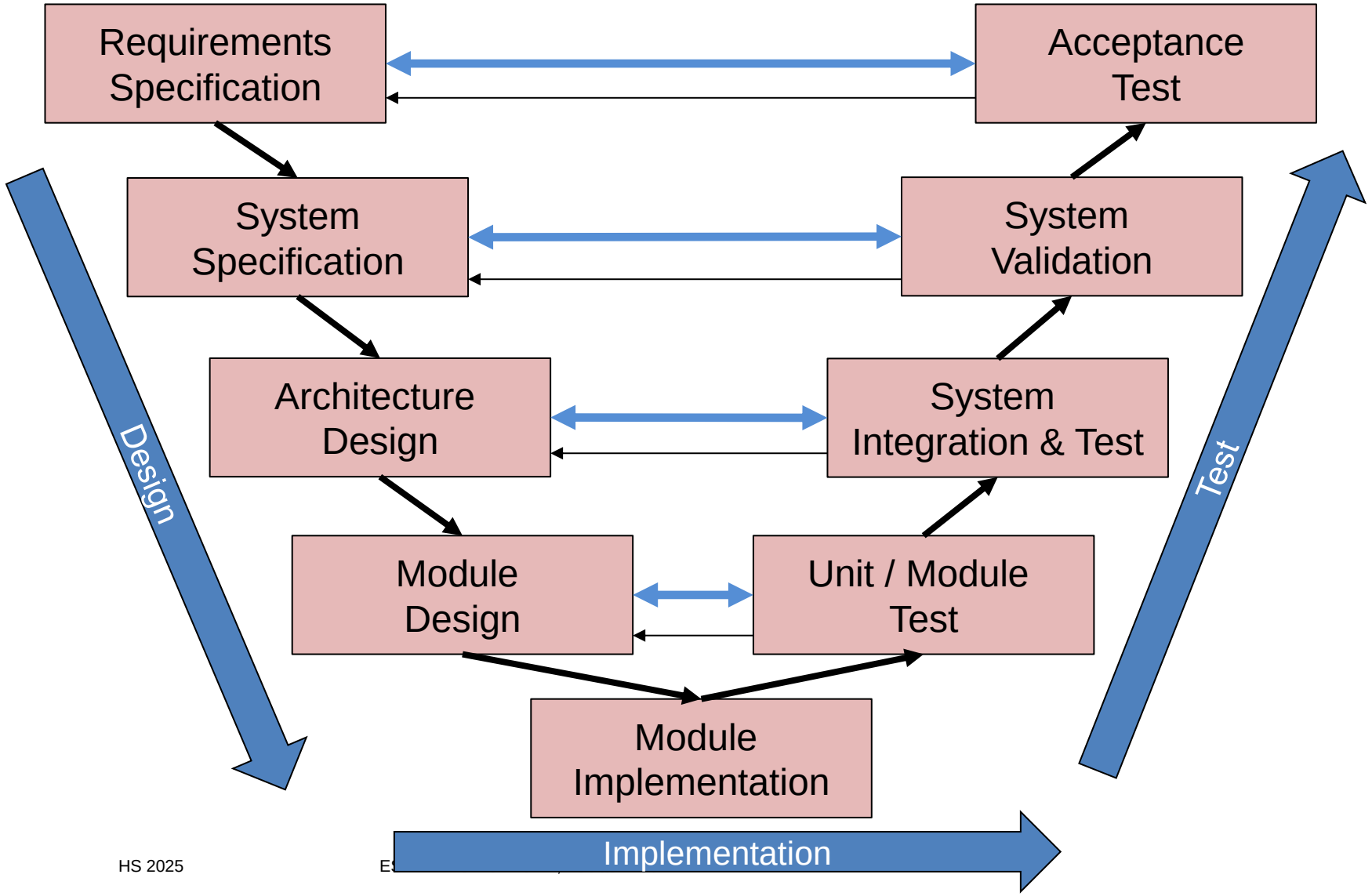
- **Next phase can only be started when the preceding phase has been completed and verified**
- **There exist many variations with slight adaptations to the phases in literature**
- **Well suited for projects that contain little uncertainty**
- **We can only see late in the project if everything works as expected**

Advantages and Disadvantages of the Waterfall Approach

- Pro‘s
  - Process is easy to understand, phases can be clearly distinguished
  - Estimation of time-to-completion is simple as all the work ahead is known and planned
  - Clear dependencies between the phases of the process
  - As there is no overlap between the phases, resource planning is easy
  - Well suited for HW development

- Con‘s
  - Strict sequential phases lack the flexibility to cope with unforeseen events or adaptations during the project
  - Users/Customers can see their final product after the whole project has been finished
  - Limited transparency on the project might cause a mismatch between the product and the expectations of the customer/user
  - Late changes in the project may cause high costs and delay
  - For SW development, the waterfall approach is too strict

V -Model



- **Well known from IT system design**
- **Design follows top-down approach**
- **Test starts bottom-up**
- **Modules can be SW or HW**
- **Iterations when needed**

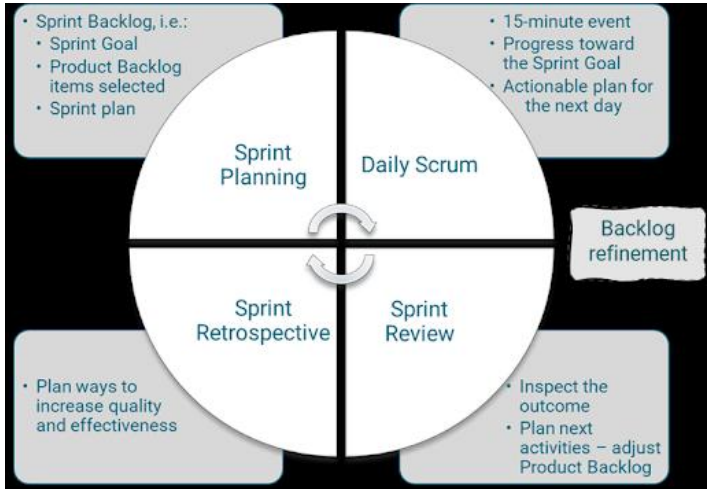
Verification of Models already during design phases

Discussion

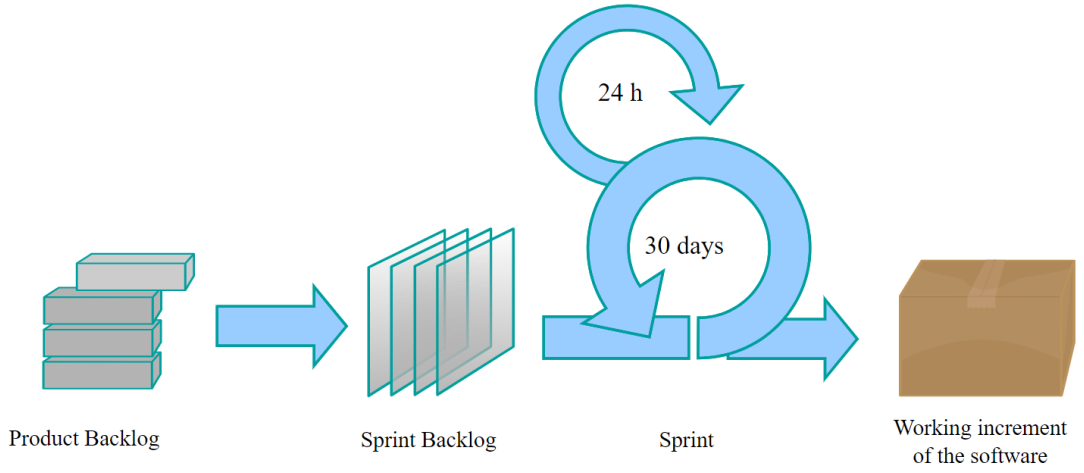
- **What development methods did you experience in your professional life?**

Agile Development Methods

- Especially for SW development, the strict waterfall model proved to be sub-optimal
- History of agile development methods for SW since 1996:
  - Test-Driven Development (Kent Beck)
  - eXtreme Programming (pair-programming, unit tests first, release frequently, etc.), see also extremeprogramming.org
  - Scrum and many variants (see next Slide)



By Stefan Morcov - Own work. Also published in my PhD thesis: Morcov, S. (2021). Managing Positive and Negative Complexity: Design and Validation of an IT Project Complexity Management Framework. PhD thesis, KU Leuven University https://lirias.kuleuven.be/retrieve/637007}, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=113548600

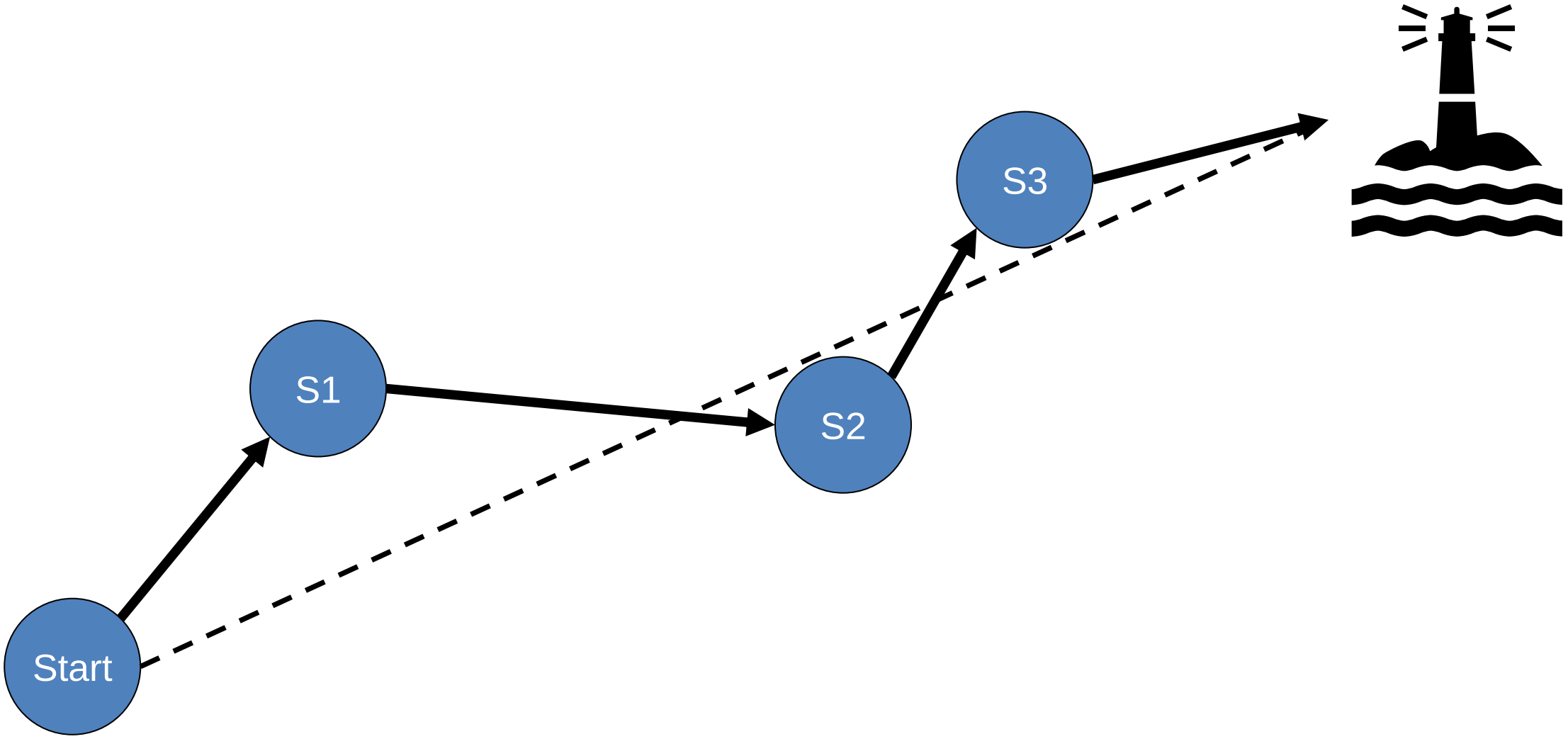


Agile Methods (a brief check on the Web…)

- **Scrum:** This agile method focuses primarily on managing tasks in team-based development conditions. In the Scrum Agile model, the team should strictly follow a work plan for each Sprint. Moreover, people involved in this type of project have predefined roles.
- **Crystal:** Using Crystal methodology is one of the most straightforward and most flexible approaches to developing software, recognizing that each project has unique characteristics. Therefore, policies and practices need to be tailored to suit them.
- **Dynamic Software Development Method (DSDM)**: This Rapid Application Development (RAD) approach involves active user involvement, and the teams are empowered to make decisions with the goal of frequent product delivery.
- **Feature Driven Development (FDD):** This Agile method focuses on "designing & building" features. It is divided into several short phases of work that must be completed for each feature separately. It includes domain walkthrough, design inspection, code inspection, etc.
- **Lean Software Development:** This methodology is based on the principle of "Just-In-Time Production." It helps to increase the speed of software development and decrease costs.
- **Extreme Programming (XP):** Extreme Programming is a useful Agile model when there are constantly changing requirements or demands from clients. It is also used when there is no sure about the system's functionality.
- **Demo Driven Development**: Demo-driven development is a practice where you use (1) regular demos, (2) a standard week-by-week project plan, and (3) value-based user stories.

Source: https://www.guru99.com/agile-model.html

Communalities of Agile Development



Advantages and Disadvantages of Agile Model

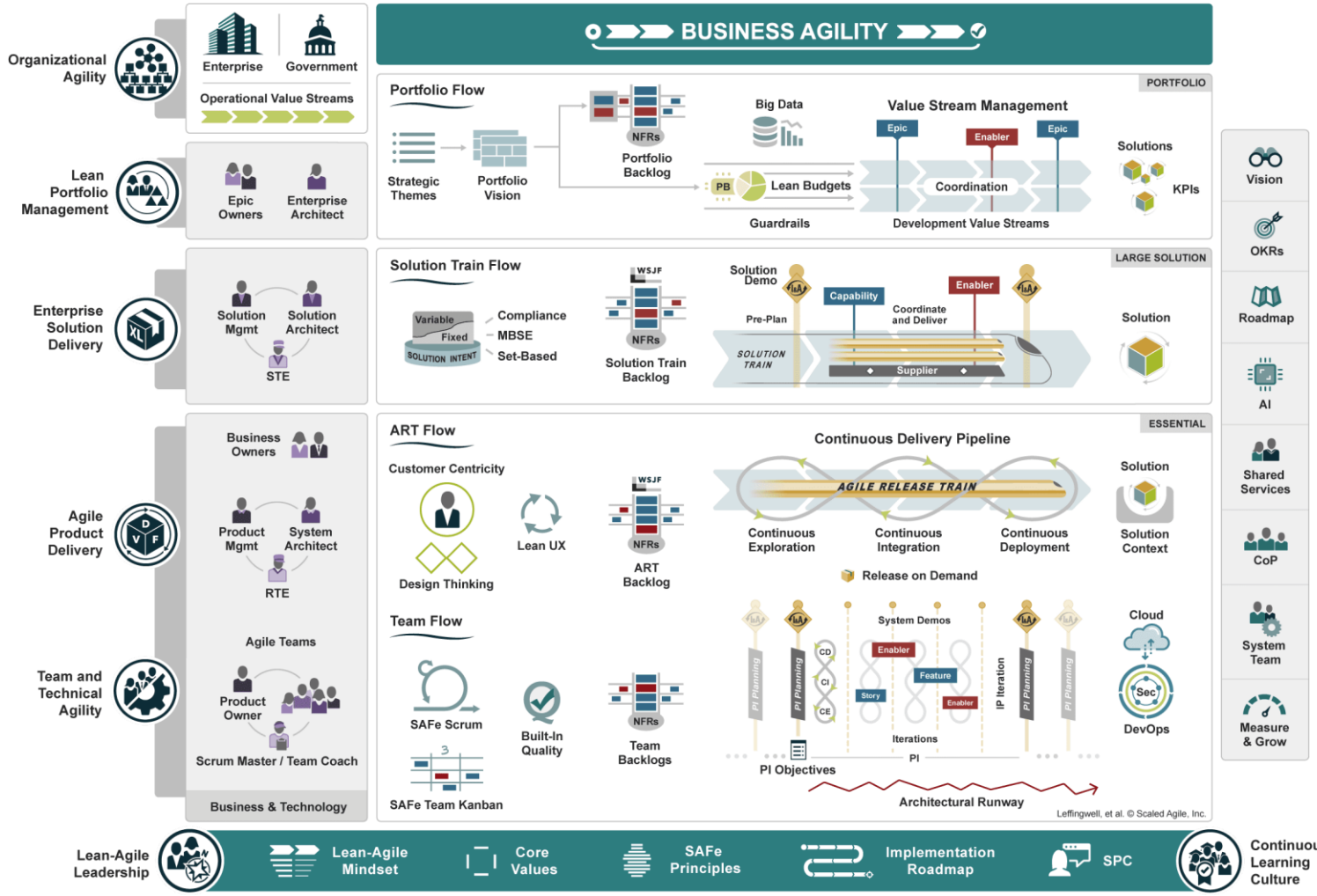
- **Advantages of the Agile Model**
  - Updated versions of functioning software are released after each sprint
  - Early feedback from users/customers on demonstrated prototypes
  - It delivers early partial working solutions.
  - Changes are acceptable at any time
  - All development team members know what others are working on through regular communication (alignment meetings, demos, etc.)

- **Disadvantages of Agile Model**
  - There is a higher risk of sustainability, maintainability, and extensibility.
  - Documentation and design are not given much attention.
  - Without clear feedback from the customer, the development team can be misled
  - Difficult to visualize and handle complex dependencies.
  - Time-to-Completion is hard to estimate

Agile Models vs. Waterfall Model

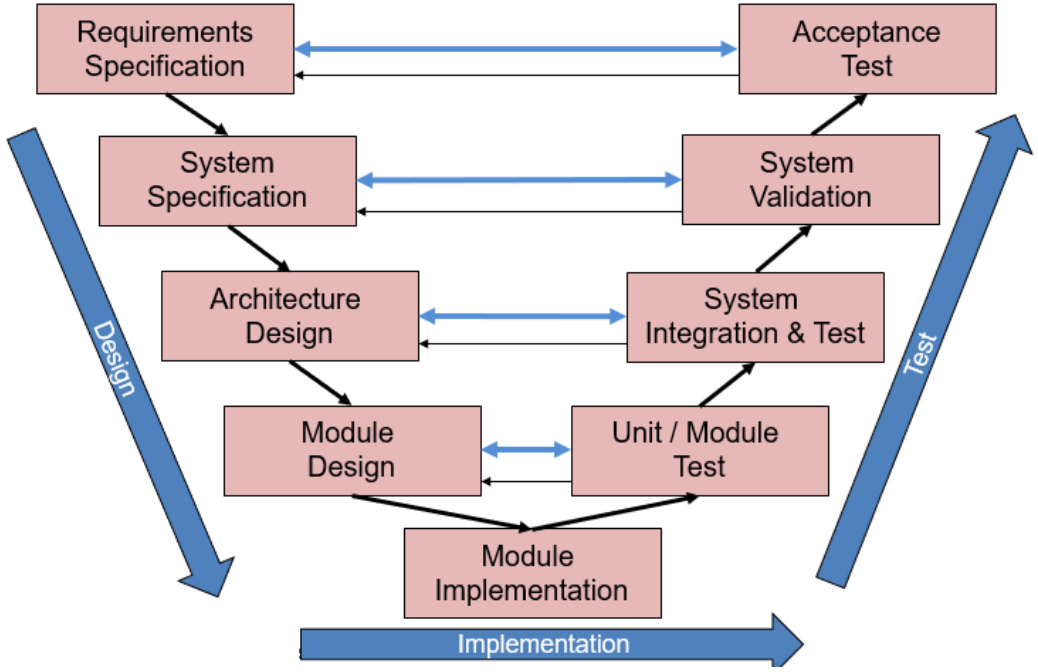
Agile Models	Waterfall Model
Agile methodologies propose incremental and iterative approaches to software design	Software development flows sequentially from start point to end point.
The development model can be individually adapted into individual models that teams work on	The design process is clear for all project team members
The customer has early and frequent opportunities to look at the product and make decisions and changes.	The user/customer can only see the product at the end of the project.
The Agile Model is considered unstructured compared to the waterfall model, and it is hard to keep the overview.	Waterfall models are more predictable because they are plan-oriented
Small projects can be implemented very quickly. For large projects, it isn't easy to estimate the development time (or time-to-completion).	All sorts of the project can be estimated and completed.

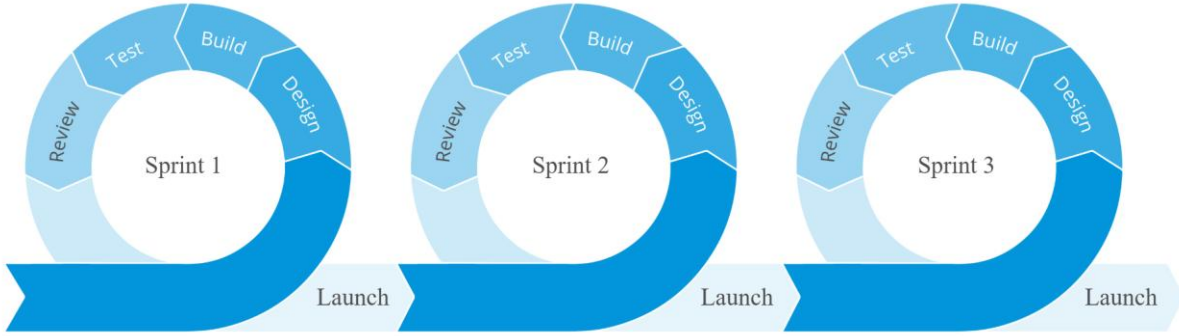
«Agile» even for Whole Enterprises



Continuous Integration / Continuous Deployment (CI/CD)

- **The V-Model and the agile methods share that they make use of feedback during the development process**





Source: www.mendix.com

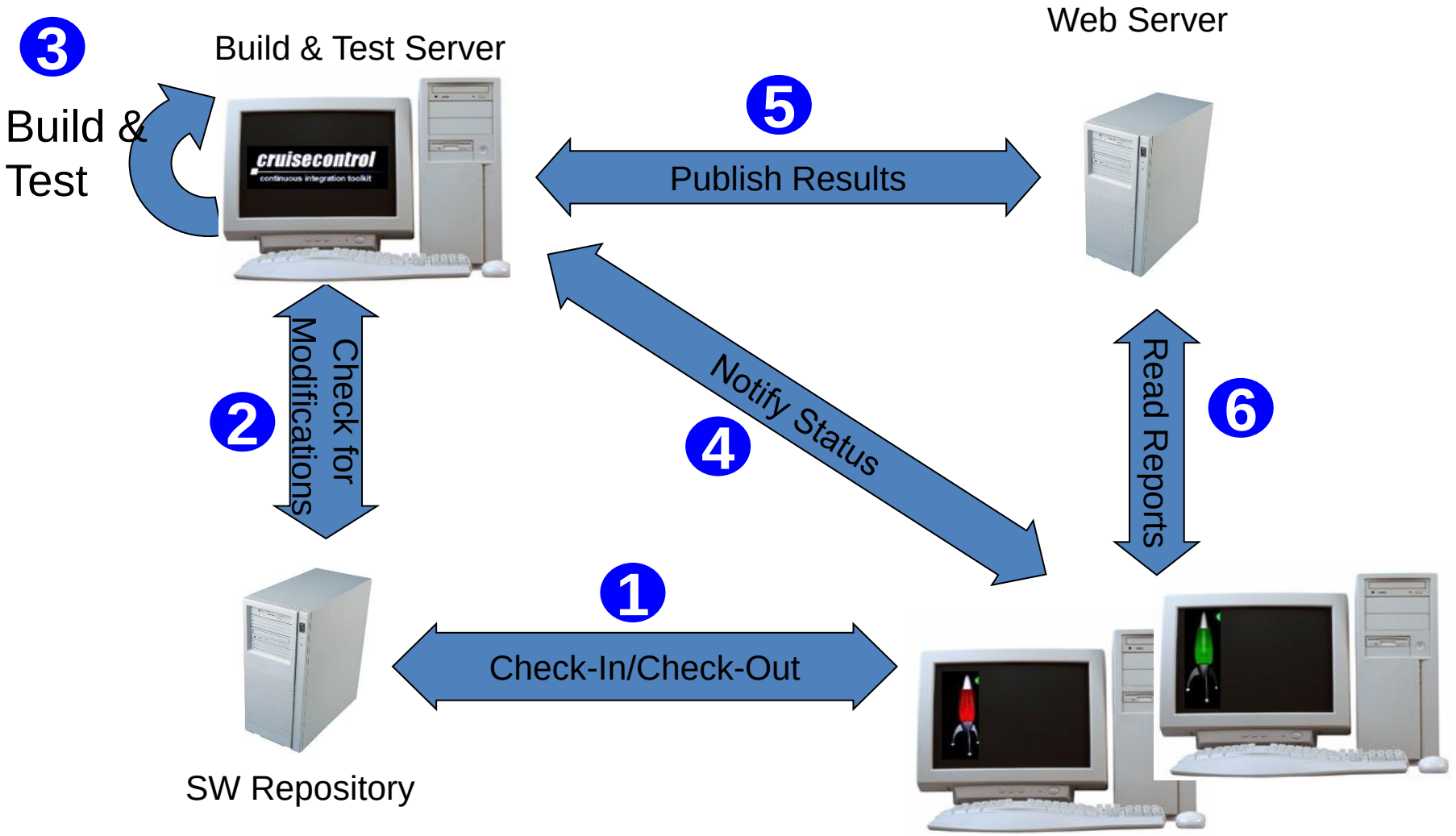
Continuous Integration / Continuous Deployment (CI/CD)

- **How can we get fast feedback?**

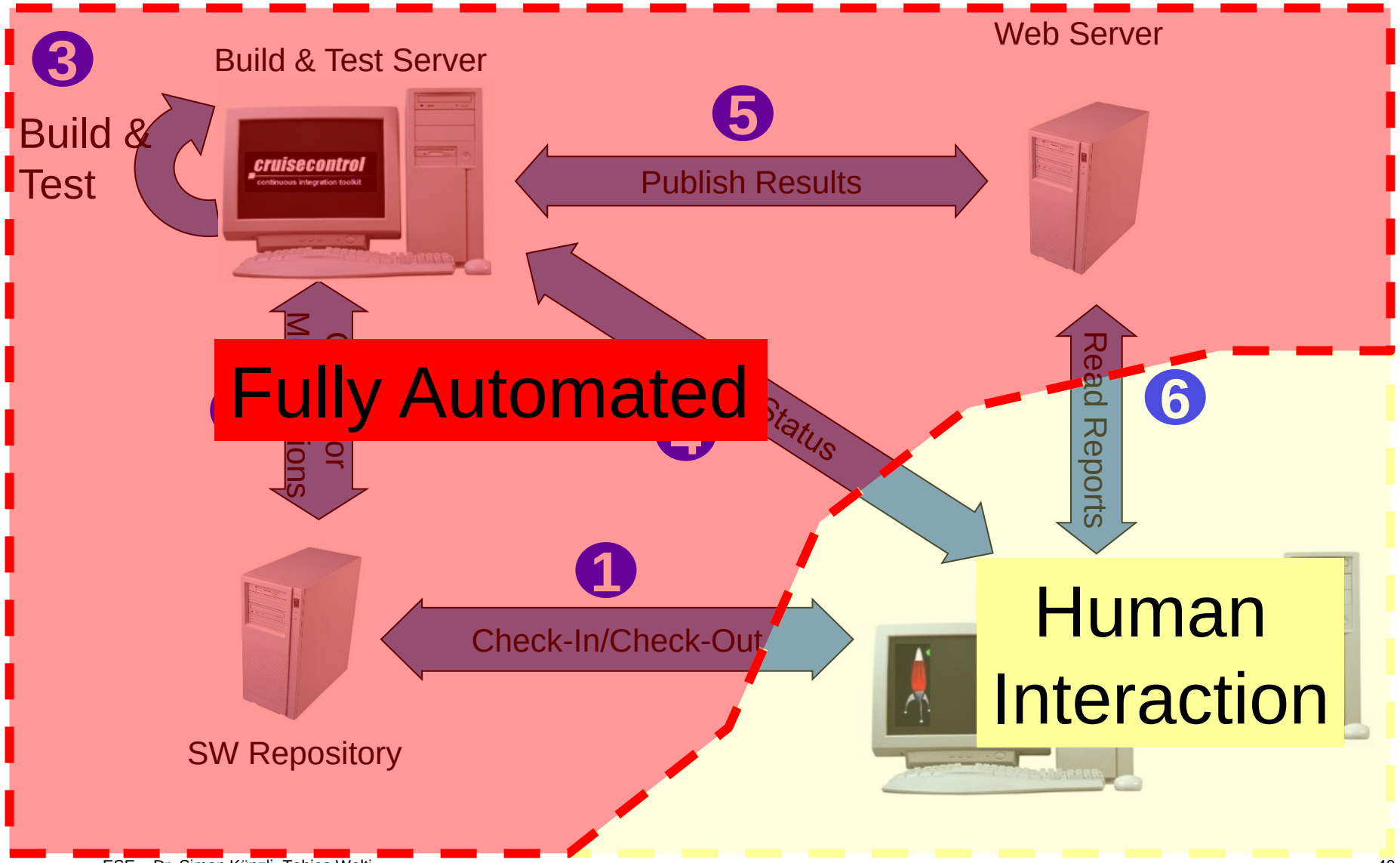
- **Integrate and test continuously**
- **Make use of CI/CD Pipelines**
  - Available Tools include:
    - Cruisecontrol
    - Hudson, succeeded by Jenkins
    - gitlabcirunner
    - github actions

- **Pipelines can be used to build, test, document, perform static code analysis, track memory footprint and a lot more for your software**
- **Pipeline triggered for every commit to the version control system**

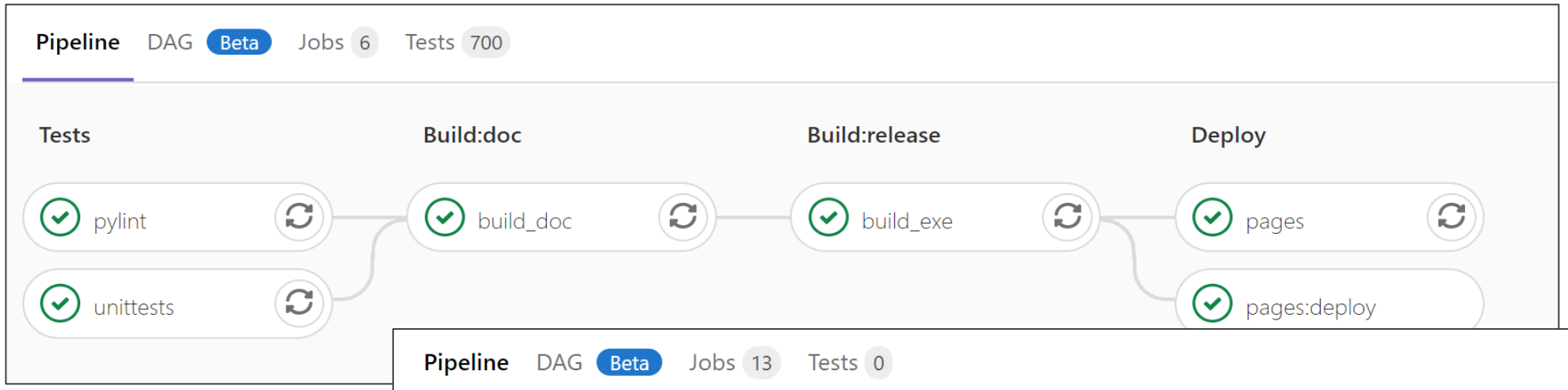
Continuous Integration: Automatic Build & Test System

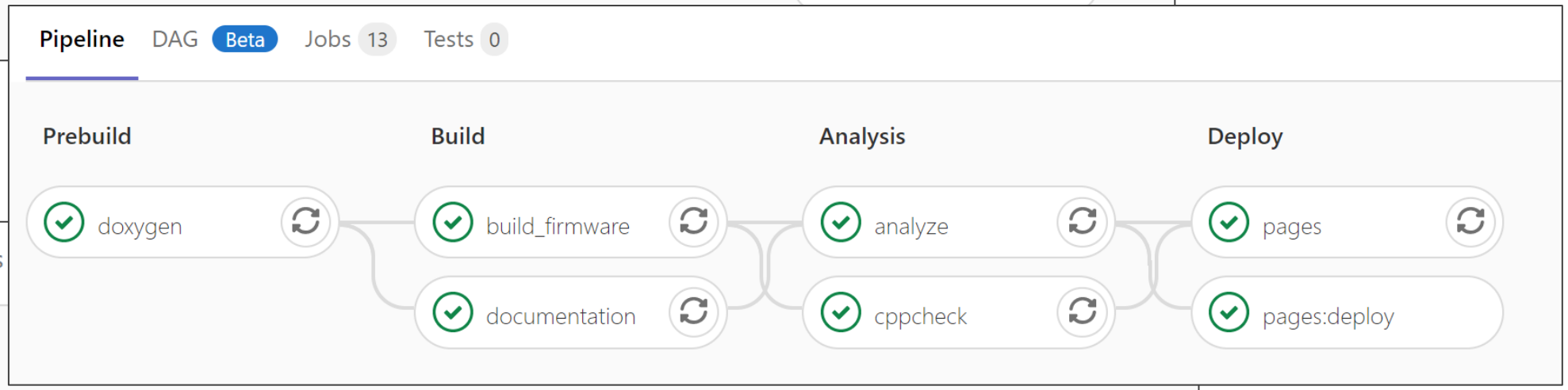


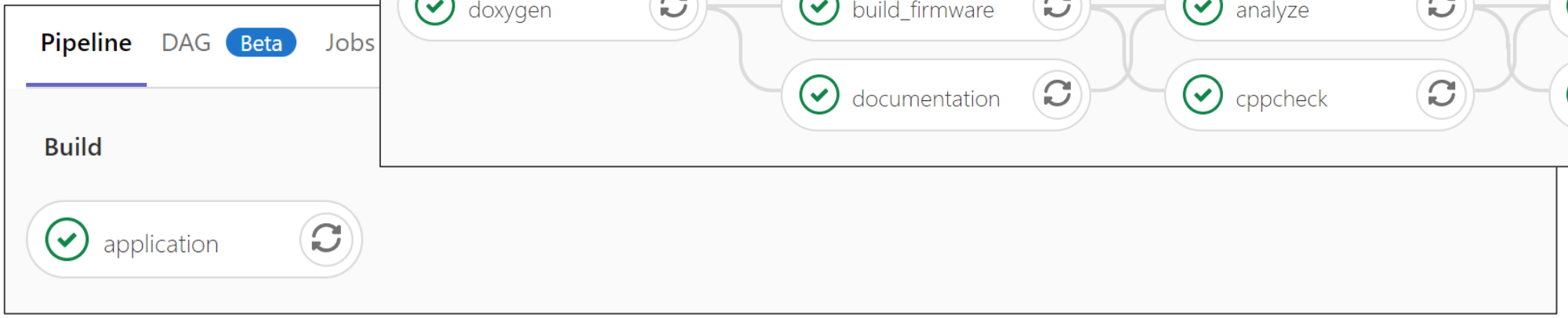
Continuous Integration: Automatic Build & Test System



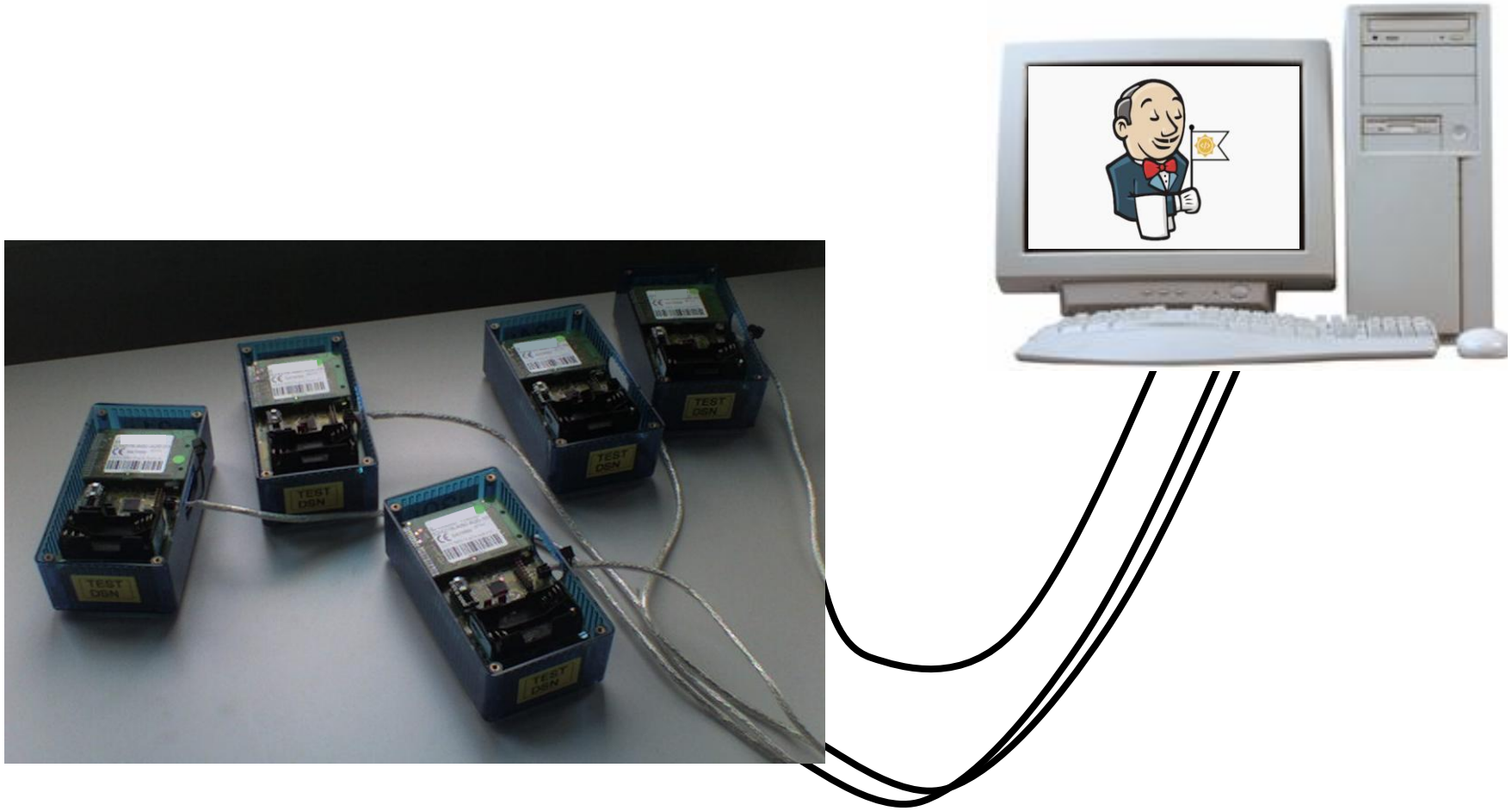
CI/CD: Example Pipelines







CI/CD: Tests with HW in-the-loop possible



As always… it depends

- **There is no “golden rule” on which development process suits best for a given development project**
- **Companies often have their development processes well defined and train their employees. The company’s general culture might influence the chosen development process, too.**
- **The overall development process may depend on the target market for your ES.**
  - E.g., medical devices, safety devices need special documentation
- **Depending on your product, the design and development phase might be short compared to the timespan between the initial idea and the market entry.**

Feedback

Zürcher Hochschule
für Angewandte Wissenschaften

School of Engineering
InES Institute of Embedded Systems

How did you perceive the first lessons (lectures)?

Thank you for your participation!

Summary

- **Specification of Embedded Systems (HW and SW) is best done using several abstraction levels**
- **Usually, Embedded Systems are designed following a top-down approach**
- **The Waterfall development process is well-suited for HW development but comes with drawbacks for SW development**
- **The V-model is widely used in industries for Embedded Systems development, whereas agile methods are targeted towards pure SW projects**